INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

LAB 2 - OPNSENSE FIREWALL POLICY TESTING, LOGGING AND PACKET ANALYSIS

WADF105 - NETWORK SECURITY FUNDAMENTALS

BY

MUSTAPHA ABUBAKAR

ID: C11/26/FCDF/17172

COHORT: FCDF-COHORT-11

PRACTICAL LABORATORY REPORT

INSTRUCTOR

MR. AMINU IDRIS

Date of Submission 06 OCTOBER, 2026

INTRODUCTION

This report documents a controlled firewall policy test carried out on the small protected network commissioned in Laboratory 1 for the International Cybersecurity and Digital Forensics Academy. The laboratory uses two Oracle VirtualBox machines: icdfa-nslab-firewall-v1, which runs OPNsense, and icdfa-nslab-client-v1, an Ubuntu workstation. A working baseline was recorded first. Two narrow temporary block rules were then created on the OPNsense LAN interface, one for ICMP to 1.1.1.1 and one for outbound HTTP on TCP port 80, and their effect was tested from the Ubuntu client. The decisions were then read in the OPNsense live log, matched with a Wireshark capture and with the state table, and the laboratory was finally restored to its original working condition. All activity remained inside the authorised ICDFA virtual laboratory.

AIM

The aim of this lab is to create and test narrow, correctly ordered firewall rules on OPNsense, to prove their effect with before-and-after evidence from the client, the firewall log and a packet capture, and to restore normal connectivity so that the next activity starts from a known-good state.

OBJECTIVES

- Explain how top-down firewall rule processing affects traffic.
- Create and test a narrow rule without accidentally blocking all connectivity.
- Use the protocol, source, destination and port fields of a rule correctly.
- Read OPNsense firewall logs and identify the rule that blocked a packet.
- Correlate firewall decisions with Ubuntu packet-capture evidence.
- Inspect the firewall state table and the automatic outbound NAT behaviour.
- Disable the test rules and verify that normal connectivity is restored.
- Document the process and results professionally.

TOOLS AND RESOURCES USED

- Oracle VirtualBox (hypervisor)
- icdfa-nslab-firewall-v1 (OPNsense firewall and router)
- icdfa-nslab-client-v1 (Ubuntu workstation)
- Linux command-line tools: ip, ping, getent, curl
- OPNsense web interface (Firewall Rules, Live View, Diagnostics States, NAT)

- ❖ Wireshark (packet capture and analysis)

NETWORK DESIGN

The network is the one commissioned in Laboratory 1 and was not changed. The firewall sits between the untrusted side (the internet, reached through the VirtualBox NAT) and the trusted side (the internal network ICDFA-LAN). All traffic from the client passes through the firewall, which is where the rules in this laboratory are applied, as shown in Figure 1 and Table 1.

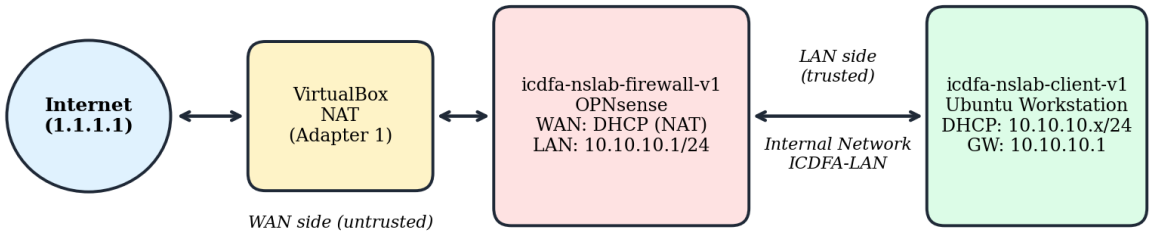


Figure 1: Two-VM network topology used in this lab

Table 1: Network design summary

Virtual machine	Adapter	Attachment	IPv4 address	Role
icdfa-nslab-firewall-v1	Adapter 1	NAT	DHCP from VirtualBox 10.0.2.15/24	OPNsense WAN
icdfa-nslab-firewall-v1	Adapter 2	Internal Network: ICDFA-LAN	10.10.10.1/24	OPNsense LAN and client gateway
icdfa-nslab-client-v1	Adapter 1	Internal Network: ICDFA-LAN	DHCP in 10.10.10.0/24 10.10.10.201/24	Ubuntu workstation

METHODOLOGY

PART 1 - RECORD THE BASELINE

Before any rule was created, the starting state was recorded. icdfa-nslab-firewall-v1 was started first and left running until its console menu appeared, and only then was icdfa-nslab-client-v1 started. A baseline is essential because, without it, a later failure could not be attributed to the new

rule rather than to the original network configuration. Testing would have stopped here if any baseline test had failed.

Step 1: Record the client address and default route

Command used

ip -4 -br address

ip route

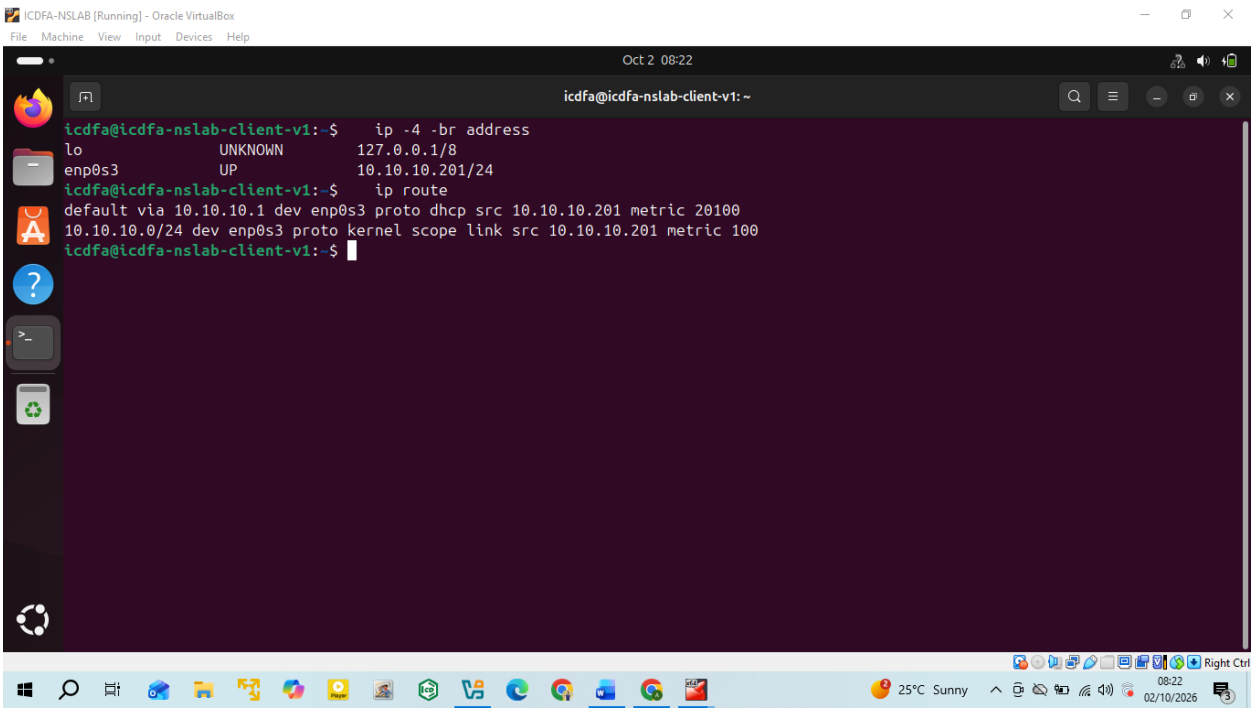


Figure 2: Baseline IPv4 address and default route on icdfa-nslab-client-v1

This step was carried out as shown in Figure 2 above. The client holds 10.10.10.201/24 on enp0s3, an address from the firewall DHCP pool, and its default route points to 10.10.10.1, the OPNsense LAN address. These values are the reference against which the later results are compared.

Step 2: Run the baseline connectivity tests

Command used

```
ping -c 4 1.1.1.1

getent hosts example.com

curl -sI http://example.com

curl -sI https://example.com
```

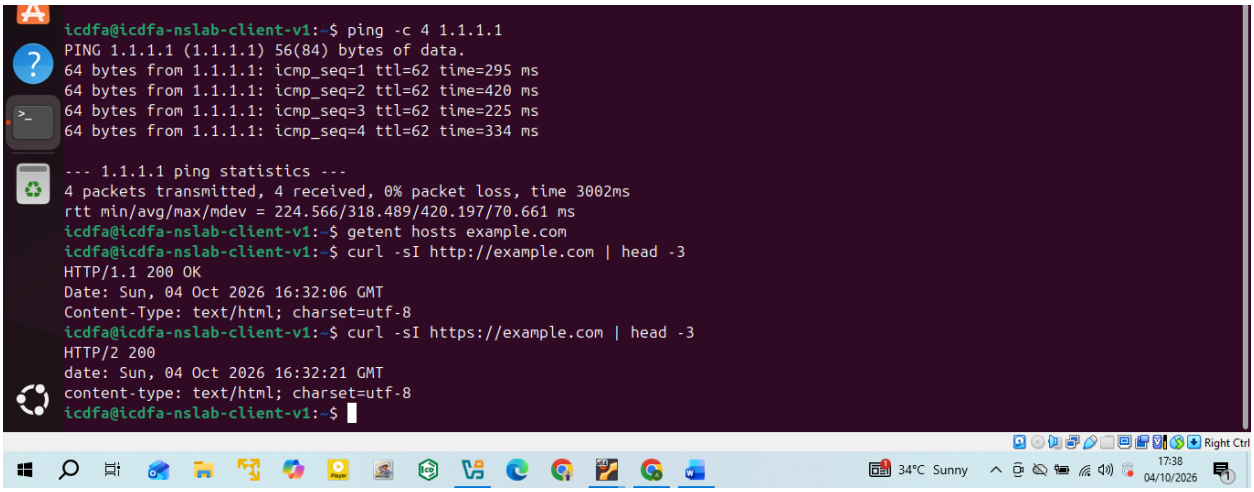


Figure 3: Successful baseline ping, DNS, HTTP and HTTPS tests

This step was carried out as shown in Figure 3 above. The ping to 1.1.1.1 returned four replies, getent resolved example.com, and the two curl commands returned HTTP response headers for both HTTP and HTTPS. Together they show that routing, NAT, DNS and both web protocols worked before any restriction was introduced.

PART 2 - REVIEW RULE ORDER

OPNsense evaluates the rules of an interface from top to bottom. By default, the first rule that matches a packet decides its fate when the rule is marked Quick, and that is the setting used for the rules in this laboratory. A specific block rule therefore has to sit above the broad allow rule to take effect. The default rule was not deleted; the temporary rules were added above it.

Step 3: Locate the Default allow LAN to any rule

Command used

```
OPNsense web interface (https://10.10.10.1) > Firewall > Rules > LAN
```

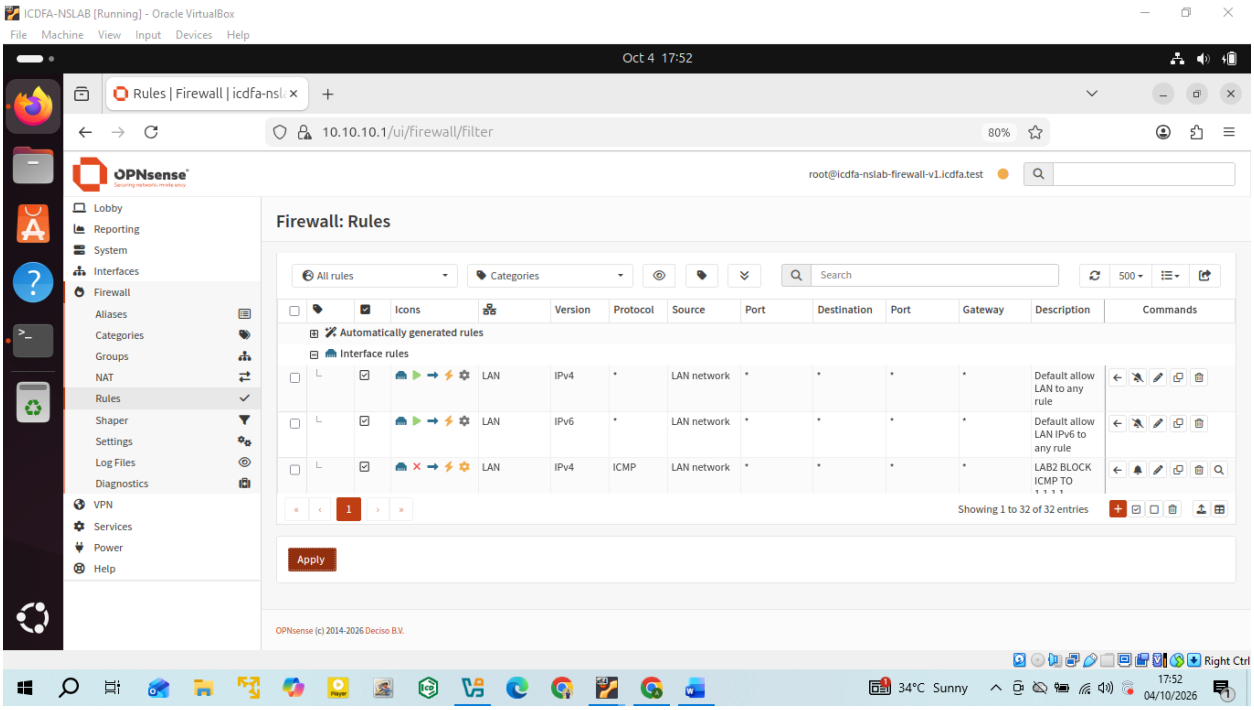


Figure 4: OPNsense LAN rules before any LAB2 rule was created

This step was carried out as shown in Figure 4 above. The LAN rules page lists the Default allow LAN to any rule, which permits the client to reach external destinations. Because it matches everything from the LAN network, any block rule placed below it would never be reached.

PART 3 - BLOCK ICMP TO ONE TEST ADDRESS

The first temporary rule blocks only IPv4 ICMP traffic from the protected LAN to 1.1.1.1, so that web and DNS traffic remain available. Narrowing the rule by protocol and by destination is what prevents it from blocking the whole connection by mistake. The field values are listed in Table 2.

Table 2: Settings of the rule LAB2 BLOCK ICMP TO 1.1.1.1

Field	Value used
Action	Block
Quick	Enabled
Interface	LAN
Direction	In
TCP/IP Version	IPv4

Field	Value used
Protocol	ICMP
ICMP type	Any (or Echo request)
Source	LAN net
Destination	Single host or Network: 1.1.1.1/32
Log packets handled by this rule	Enabled
Description	LAB2 BLOCK ICMP TO 1.1.1.1

Step 4: Create the ICMP block rule

Command used

```
Firewall > Rules > LAN > Add (+)  >  fields as in Table 2  >  Save  >
Apply changes
```

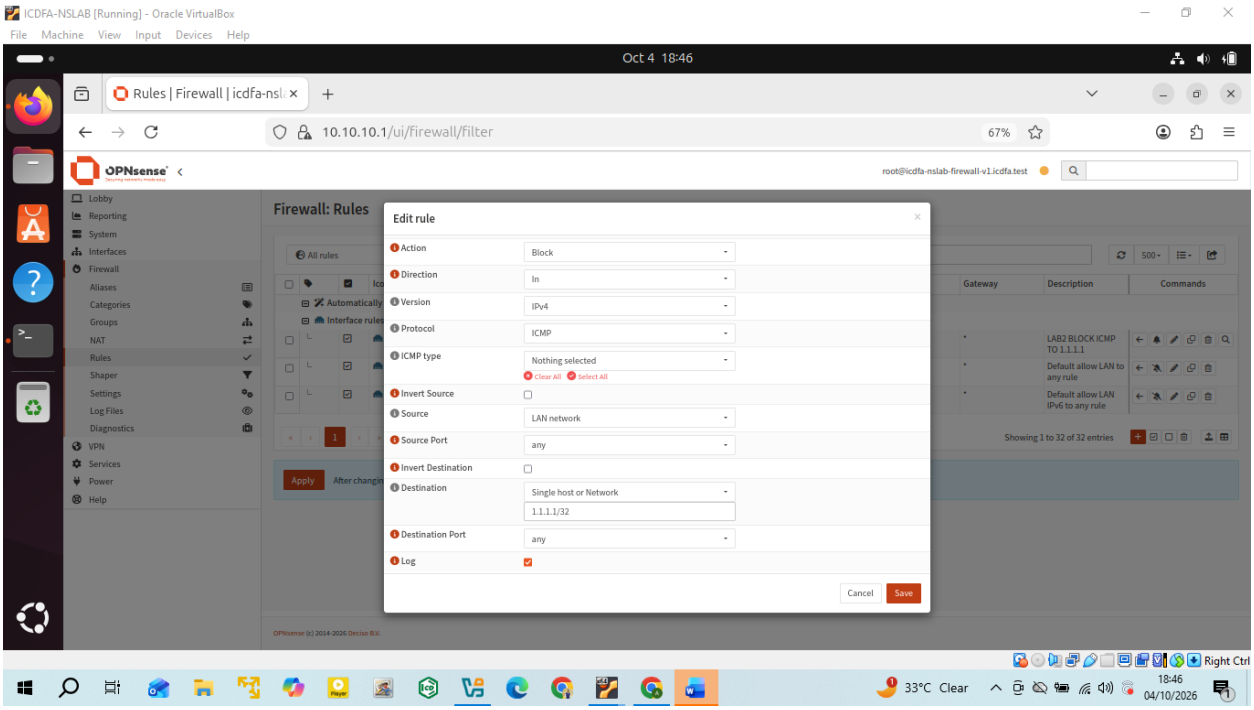


Figure 5: Rule form for LAB2 BLOCK ICMP TO 1.1.1.1

This step was carried out as shown in Figure 5 above. The rule blocks only ICMP from LAN net to the single host 1.1.1.1/32, with logging enabled so that every match is recorded. After saving, the rule was moved above Default allow LAN to any and the changes were applied.

Step 5: Test the blocked ICMP while DNS and HTTPS still work

Command used

```
ping -c 4 1.1.1.1

getent hosts example.com

curl -I https://example.com
```

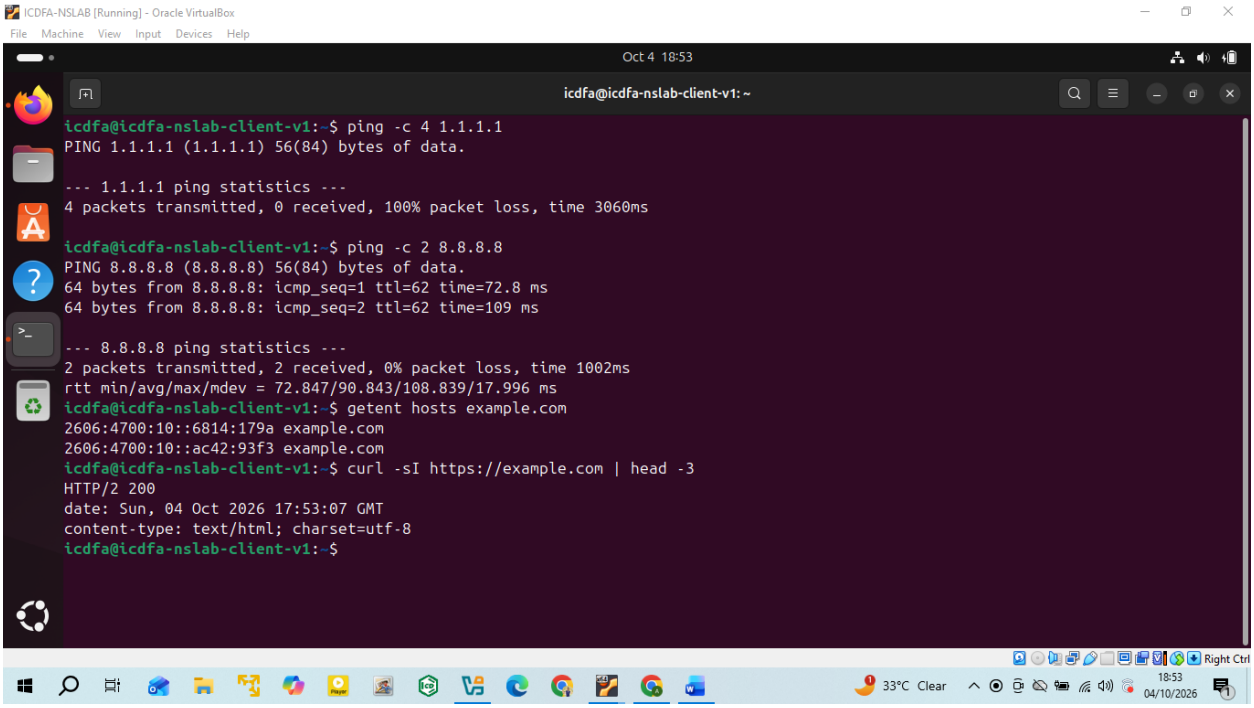


Figure 6: Failed ping to 1.1.1.1 with DNS and HTTPS still working

This step was carried out as shown in Figure 6 above. The ping to 1.1.1.1 received no replies and reported 100% packet loss, while name resolution and the HTTPS request still succeeded. A failed ping alongside successful DNS and HTTPS demonstrates protocol-specific filtering; it does not mean the whole internet connection is down.

PART 4 - BLOCK OUTBOUND HTTP WHILE ALLOWING HTTPS

The second temporary rule blocks unencrypted web traffic on TCP port 80 and leaves HTTPS on TCP port 443 untouched. It was also placed above the broad allow rule. The field values are listed in Table 3.

Table 3: Settings of the rule LAB2 BLOCK OUTBOUND HTTP

Field	Value used
Action	Block
Interface	LAN
Direction	In

Field	Value used
TCP/IP Version	IPv4
Protocol	TCP
Source	LAN net
Destination	Any
Destination port range	HTTP to HTTP (port 80)
Log packets handled by this rule	Enabled
Description	LAB2 BLOCK OUTBOUND HTTP

Step 6: Create the HTTP block rule

Command used

```
Firewall > Rules > LAN > Add (+)  >  fields as in Table 3  >  Save  >
Apply changes
```

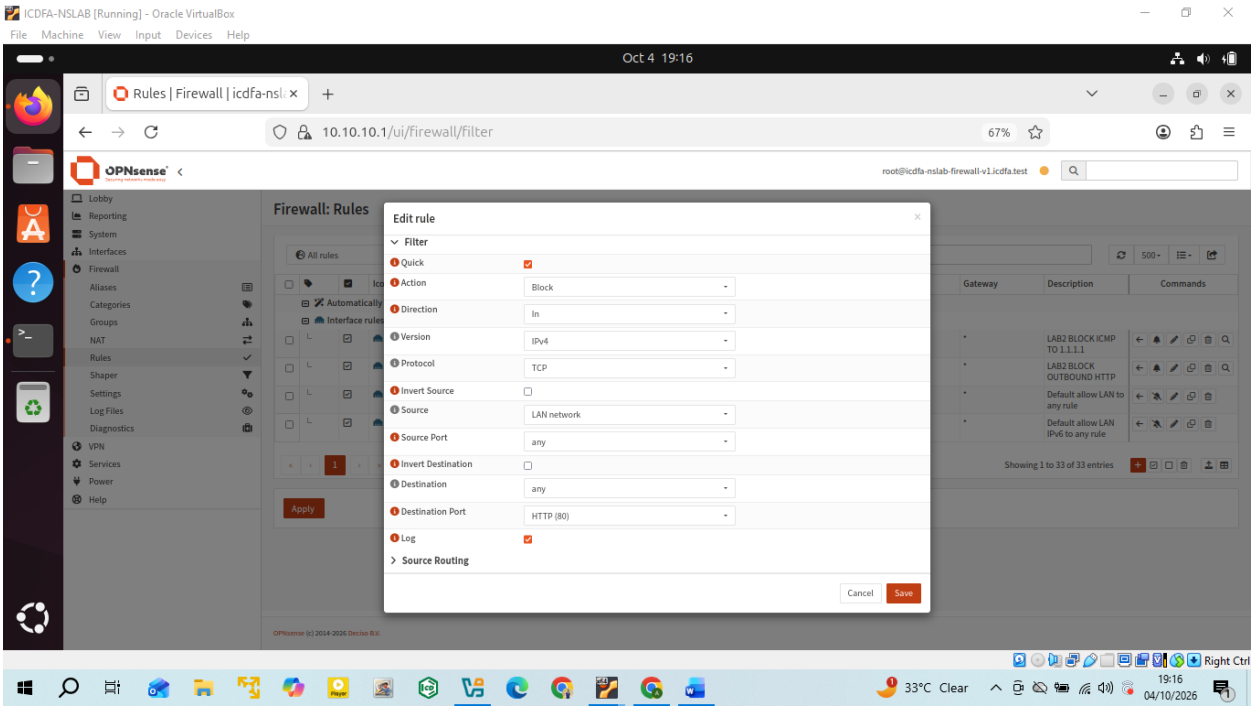


Figure 7: Rule form for LAB2 BLOCK OUTBOUND HTTP

This step was carried out as shown in Figure 7 above. The rule blocks TCP traffic from LAN net to any destination on port 80 only. It was saved, positioned above Default allow LAN to any, and applied.

Step 7: Confirm the rule order on the LAN page

Command used

```
Firewall > Rules > LAN
```

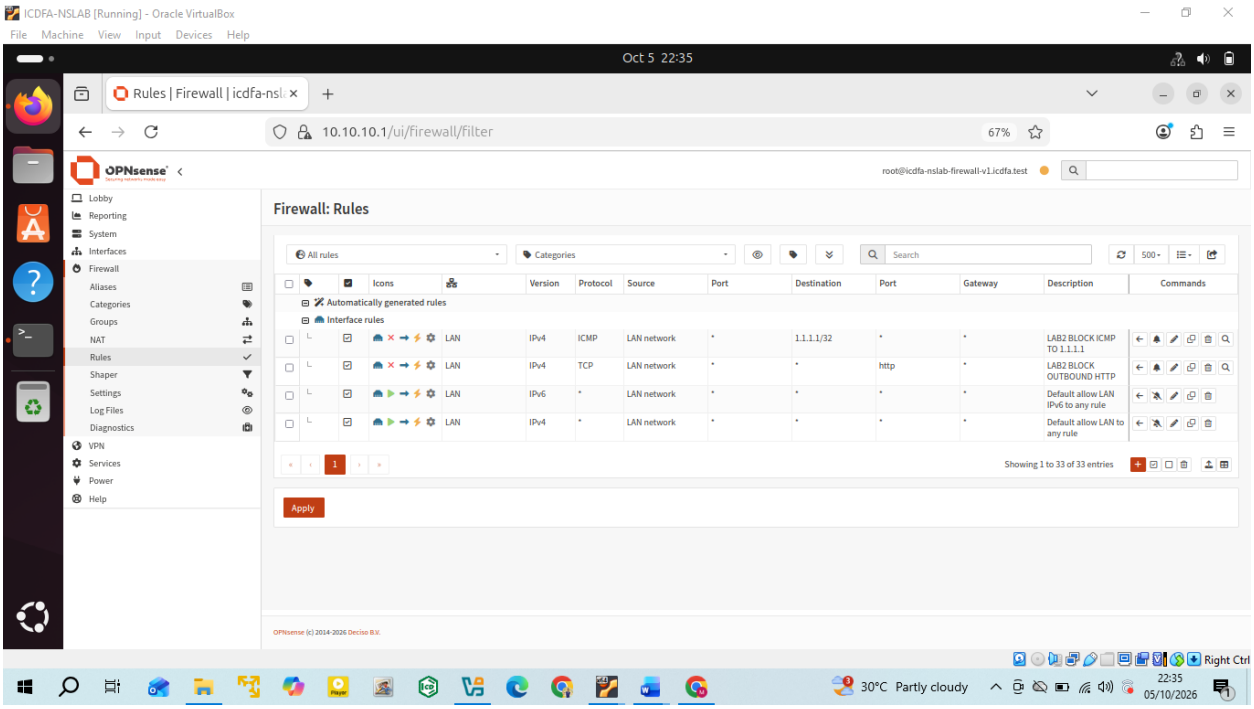


Figure 8: LAN rules showing both LAB2 block rules above the default allow rule

This step was carried out as shown in Figure 8 above. Both LAB2 block rules appear above the Default allow LAN to any rule. Because the rules are read from the top, a packet that matches a block rule is dropped before the allow rule is ever considered.

Step 8: Compare an HTTP request with an HTTPS request

Command used

```
curl --max-time 10 -I http://example.com

curl --max-time 10 -I https://example.com
```

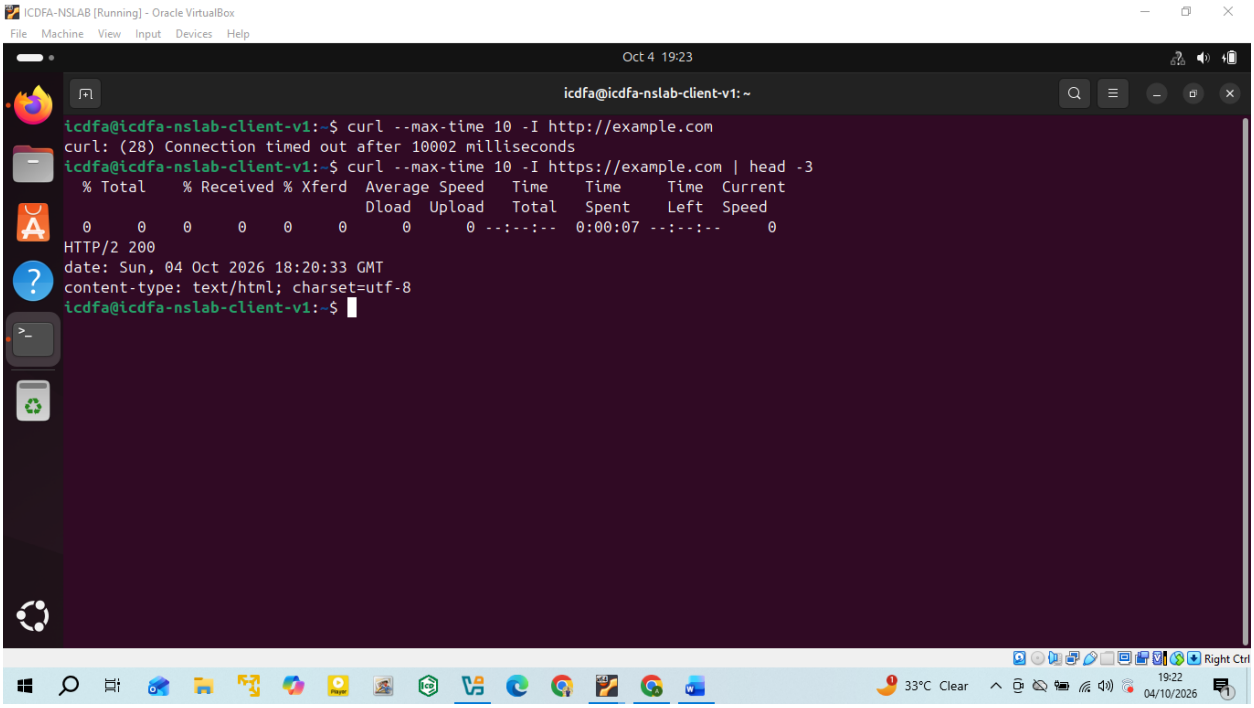


Figure 9: Failed HTTP request and successful HTTPS request

This step was carried out as shown in Figure 9 above. The HTTP request failed after the 10-second limit, and the HTTPS request still returned response headers. The -L option was not used, so a redirect could not hide the result. The firewall therefore filters by destination port: the same destination was reachable on port 443 and not on port 80.

PART 5 - EXAMINE FIREWALL LOGS

The OPNsense live log shows every packet handled by a rule that has logging enabled. Fresh blocked traffic was generated by repeating the ping and the HTTP curl command, and the log was filtered by the client address, the destination 1.1.1.1, destination port 80 and the LAB2 description.

Step 9: Generate blocked traffic and open the Live View

Command used

```
ping -c 4 1.1.1.1

curl --max-time 10 -I http://example.com

OPNsense: Firewall > Log Files > Live View
```

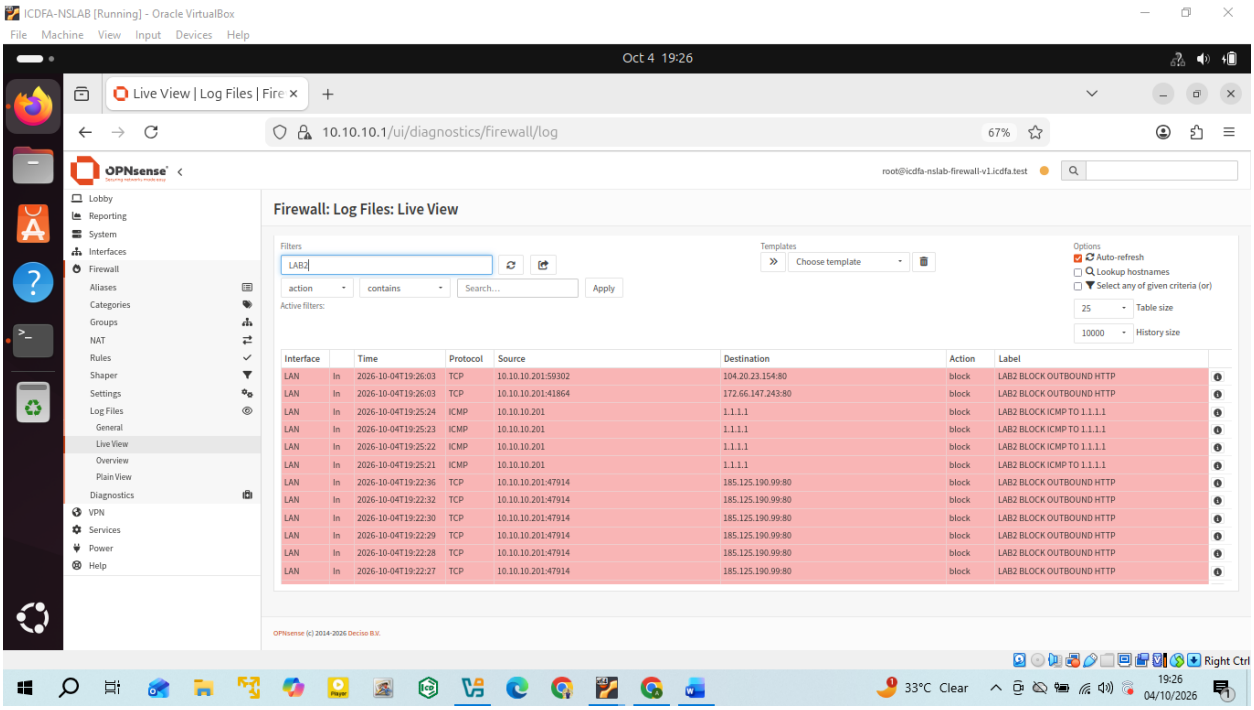


Figure 10: OPNsense Live View entries for blocked ICMP and TCP port 80 traffic

This step was carried out as shown in Figure 10 above. The Live View shows blocked entries from the client for ICMP to 1.1.1.1 and for TCP to port 80, each labelled with the LAB2 rule that handled it.

Step 10: Record the details of the matching entries

Command used

```
Live View > click the information icon of a matching entry
```

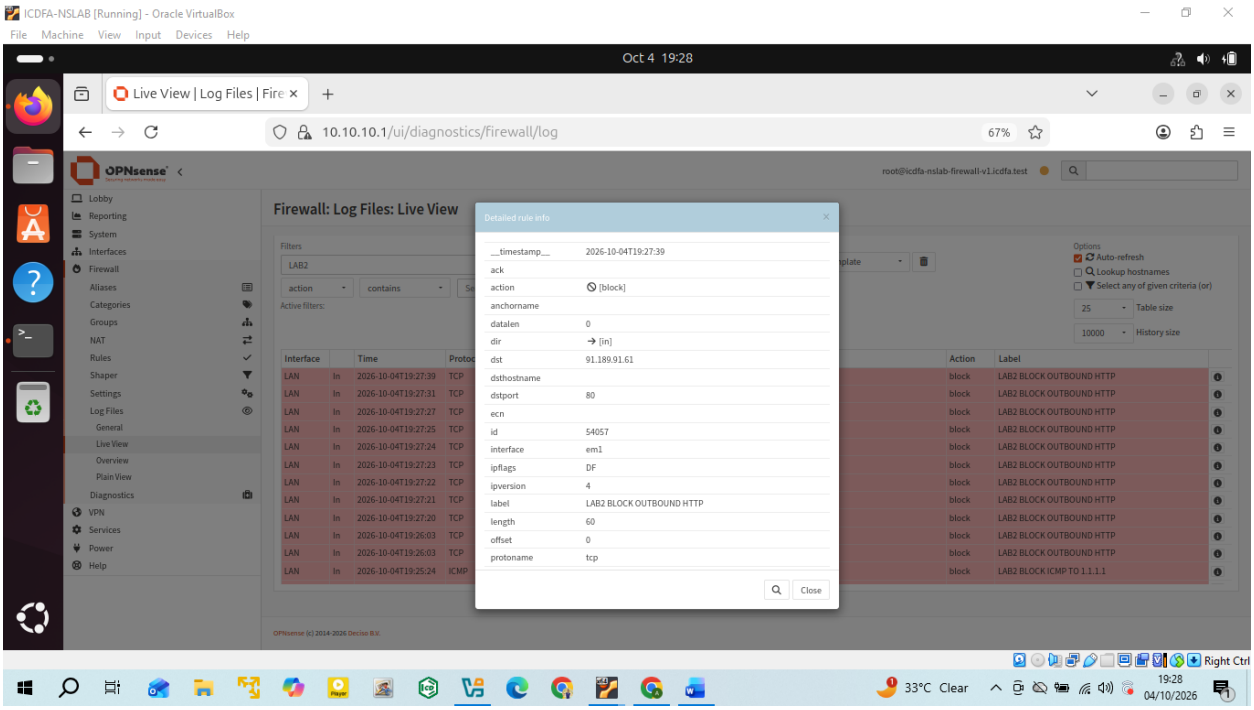


Figure 11: Detail view of a blocked log entry

This step was carried out as shown in Figures 10 and 11 above and the values are recorded in Table 4. Figure 11 shows the detail of another blocked port 80 entry (19:27:39, destination 91.189.91.61, interface em1) handled by the same rule. Each packet matched a block rule because the block rules sit above Default allow LAN to any and their conditions (protocol, source, destination and port) all matched, so the evaluation ended at the block rule and the allow rule was never reached.

Table 4: Recorded log evidence

Field	Blocked ICMP entry	Blocked HTTP entry
Timestamp	2026-10-04 19:25:24	2026-10-04 19:26:03
Interface	LAN	LAN
Source IP	10.10.10.201	10.10.10.201
Destination IP	1.1.1.1	104.20.23.154 (example.com)
Protocol	ICMP	TCP
Destination port	Not applicable	80
Action	Block	Block
Rule label	LAB2 BLOCK ICMP TO 1.1.1.1	LAB2 BLOCK OUTBOUND HTTP

PART 6 - CORRELATE WITH WIRESHARK

Wireshark was started on icdfa-nslab-client-v1 and the interface enp0s3 carrying the 10.10.10.x address was selected for capture. While capturing, ping -c 4 1.1.1.1, curl --max-time 10 -I http://example.com and curl --max-time 10 -I https://example.com were run. The capture was then stopped and each display filter was typed so that it is visible in every screenshot.

Step 11: Blocked ICMP in the capture

Command used

```
ping -c 4 1.1.1.1

Wireshark display filter:  icmp && ip.addr == 1.1.1.1
```

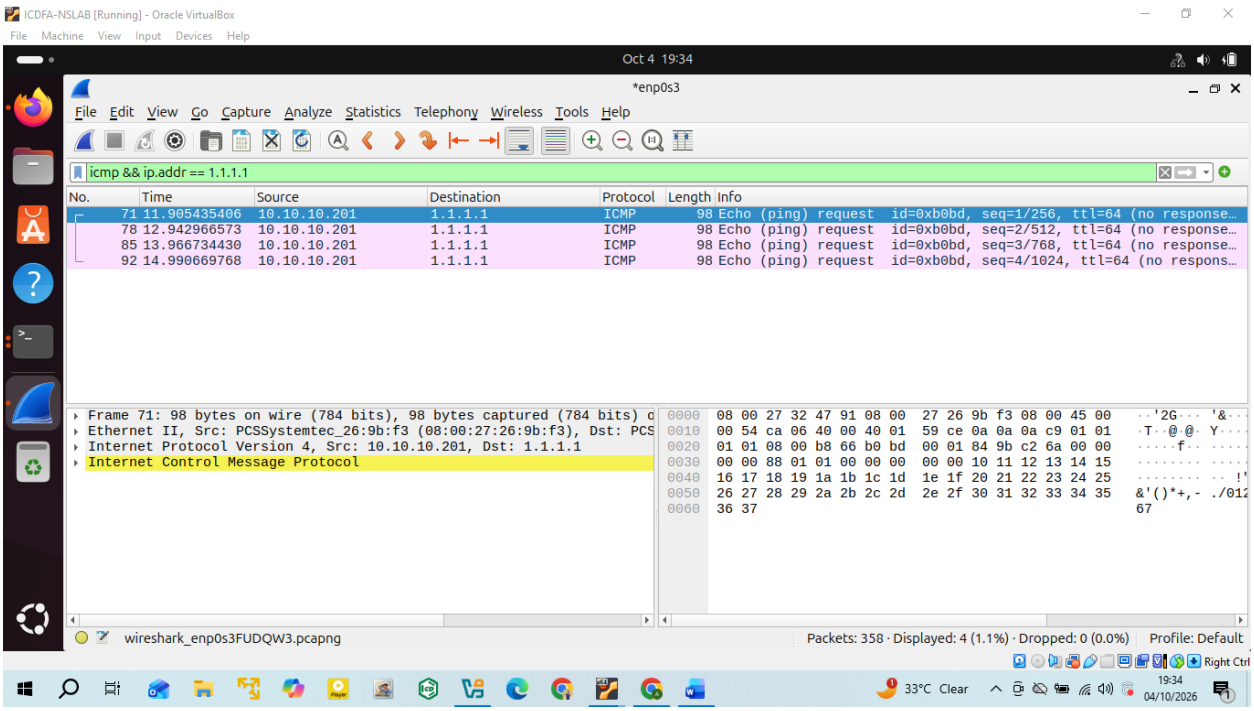


Figure 12: Wireshark view of ICMP to 1.1.1.1 with no replies

This step was carried out as shown in Figure 12 above. The client sent echo requests to 1.1.1.1, but no echo replies came back. The firewall dropped the requests silently, so the packets leave the client but the exchange is never completed.

Step 12: Blocked HTTP in the capture

Command used

```
curl --max-time 10 -I http://example.com

Wireshark display filter:  tcp.dstport == 80
```

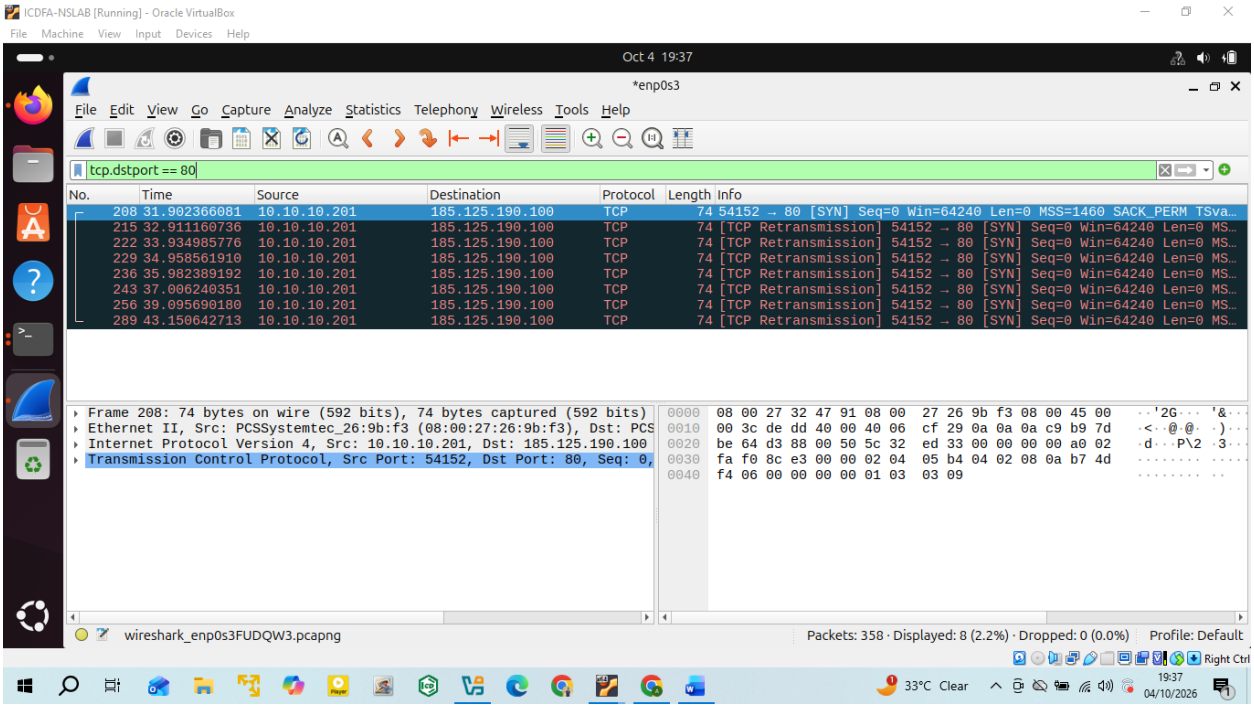


Figure 13: Wireshark view of TCP port 80 SYN packets with no handshake

This step was carried out as shown in Figure 13 above. The client sent a TCP SYN to port 80 (here to 185.125.190.100, another port 80 connection that the same rule blocked) and received no SYN-ACK. The SYN was retransmitted several times because the firewall silently blocks it, so the three-way handshake never completed and no HTTP data was exchanged.

Step 13: Permitted HTTPS in the capture

Command used

```
curl --max-time 10 -I https://example.com

Wireshark display filter:  tcp.port == 443
```

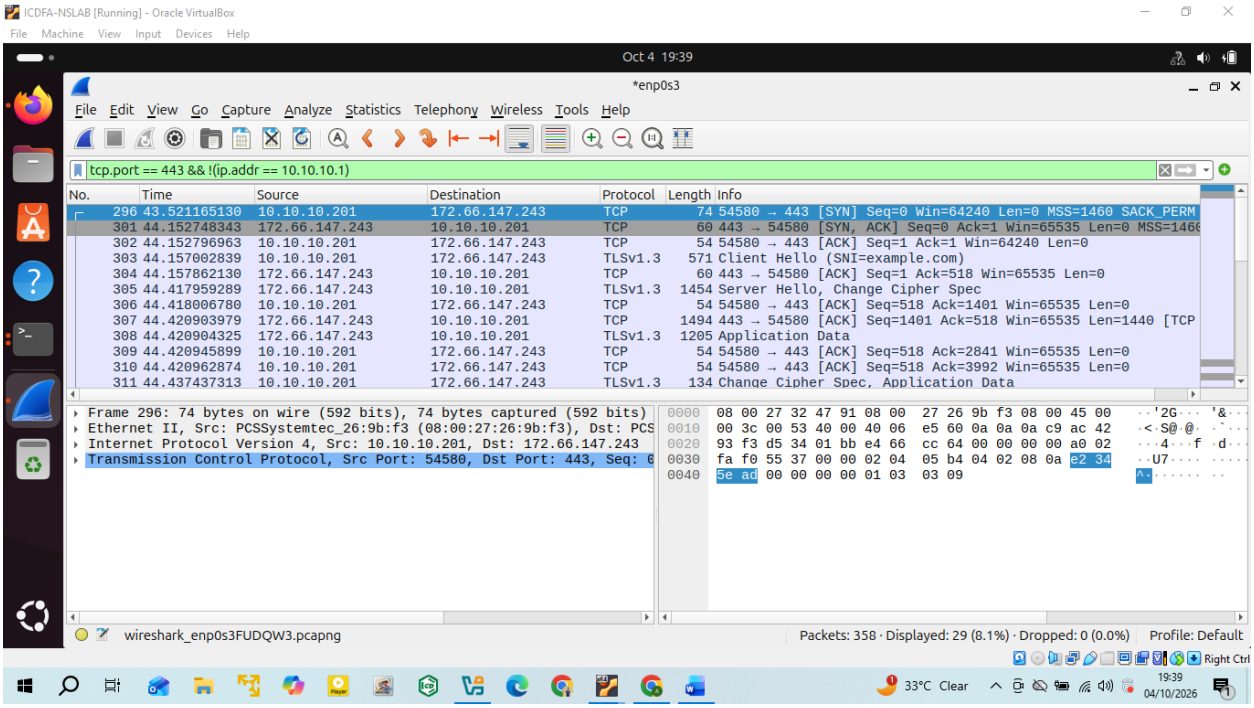


Figure 14: Wireshark view of a completed HTTPS connection

This step was carried out as shown in Figure 14 above. The permitted HTTPS connection shows a complete TCP handshake (SYN, SYN-ACK, ACK) followed by encrypted TLS traffic. The contrast with port 80 is the evidence: blocked traffic gets no successful response, permitted traffic completes a connection.

Table 5: Summary of the Wireshark observations

Display filter	Observation	Meaning
icmp && ip.addr == 1.1.1.1	Echo requests leave the client; no echo replies return.	Blocked by LAB2 BLOCK ICMP TO 1.1.1.1.
tcp.dstport == 80	SYN packets repeated; no SYN-ACK.	Blocked by LAB2 BLOCK OUTBOUND HTTP.
tcp.port == 443	Full handshake followed by TLS traffic.	Permitted by Default allow LAN to any.

PART 7 - INSPECT STATES AND AUTOMATIC NAT

OPNsense is a stateful firewall: when a connection is permitted, it creates a state so that the return traffic is accepted automatically. The state table was examined for a fresh HTTPS request, and the outbound NAT mode was checked without being changed.

Step 14: View the state of a permitted HTTPS connection

Command used

```
curl --max-time 10 -I https://example.com

OPNsense: Firewall > Diagnostics > States  (filter: 10.10.10.201)
```

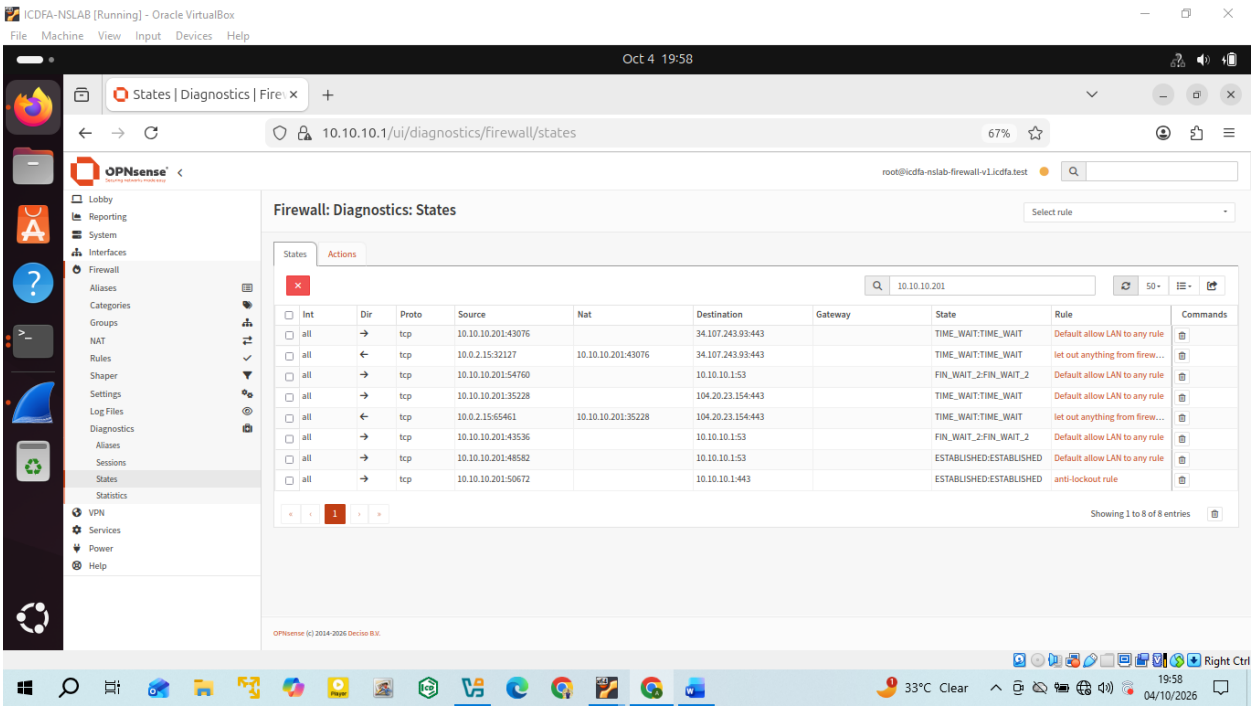


Figure 15: State table entry for a permitted HTTPS connection

This step was carried out as shown in Figure 15 above. The state table lists the connection from 10.10.10.201 to the web server on port 443 on the LAN side, and the translated state with the WAN address 10.0.2.15 as the source. The firewall uses this state to match the returning packets to the client.

Step 15: Confirm the outbound NAT mode

Command used

```
OPNsense: Firewall > NAT > Source NAT  (called Outbound in the lab brief)
```

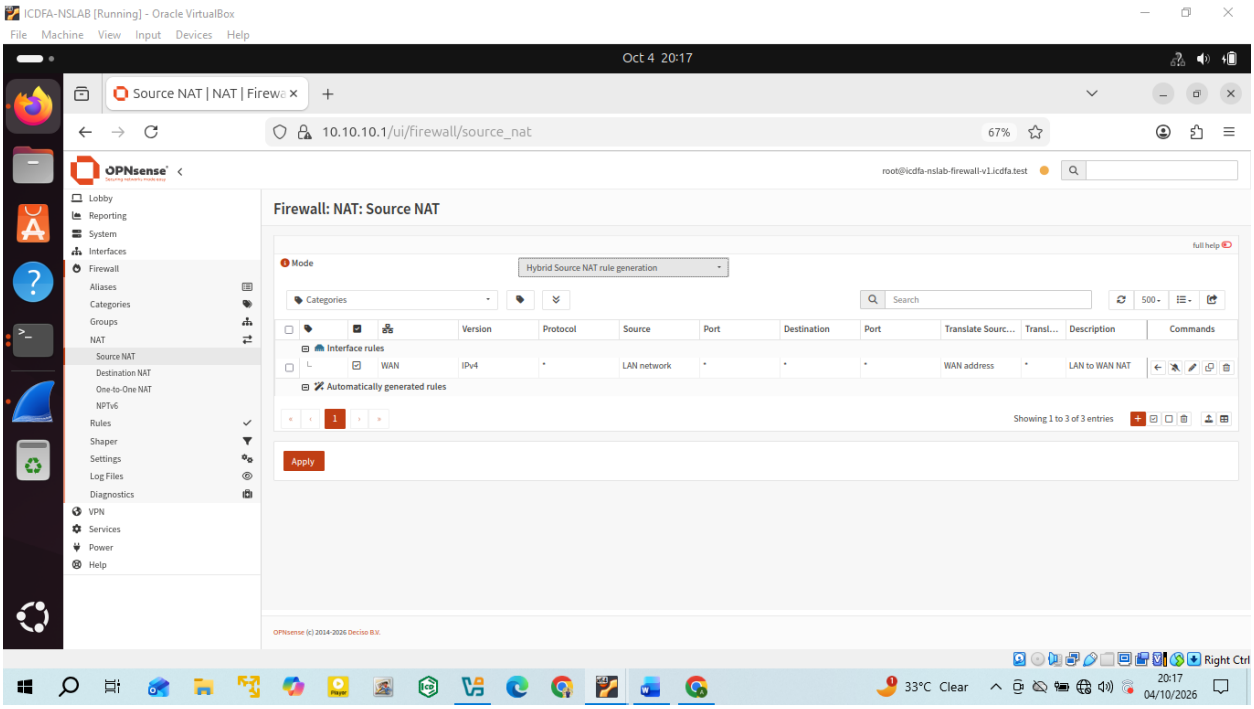


Figure 16: OPNsense outbound (source) NAT mode

This step was carried out as shown in Figure 16 above. The current mode is Hybrid Source NAT rule generation, with the rule LAN to WAN NAT (added in Laboratory 1) translating the LAN network to the WAN address. The client uses a private address, 10.10.10.0/24, which cannot be routed on the internet, so outbound NAT replaces the source address with the WAN address on the way out and restores it on the way back. The mode was not changed during this laboratory.

PART 8 - RESTORE THE LABORATORY

The two test rules were disabled but not deleted until all screenshots were complete, and the changes were applied. If old connections affected the result, only the relevant states would have been cleared. The baseline tests were then repeated.

Step 16: Disable the two LAB2 rules

Command used

```
Firewall > Rules > LAN > disable LAB2 BLOCK ICMP TO 1.1.1.1 and LAB2
BLOCK OUTBOUND HTTP  >  Apply changes
```

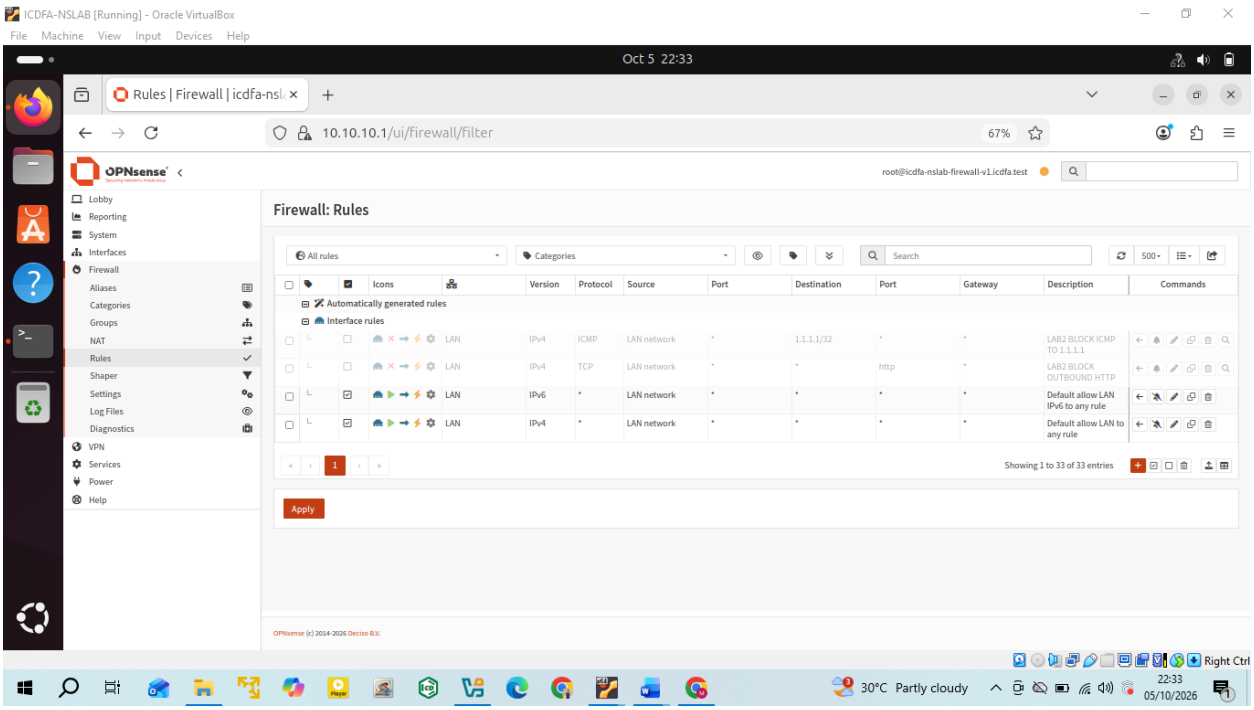


Figure 17: LAN rules showing both LAB2 rules disabled

This step was carried out as shown in Figure 17 above. Both temporary rules are disabled and the changes were applied, so the firewall returned to relying on the default allow rule. The rules were kept, not deleted, so that the evidence remains complete.

Step 17: Verify that normal access is restored

Command used

```
ping -c 4 1.1.1.1

curl --max-time 10 -I http://example.com

    curl --max-time 10 -I https://example.com
```

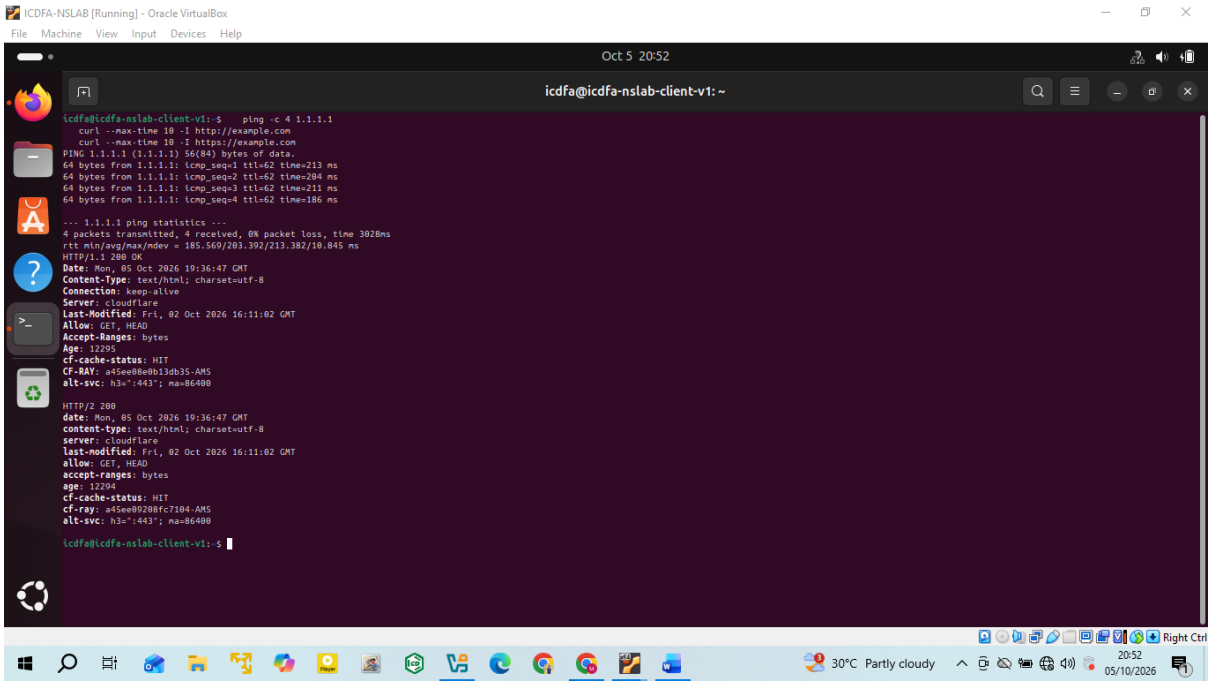


Figure 18: Restored connectivity after the LAB2 rules were disabled

This step was carried out as shown in Figure 18 above. The ping succeeded and both the HTTP and HTTPS requests returned response headers, which matches the baseline. The laboratory is therefore back in its known-good state.

DISCUSSION

The laboratory shows that a firewall rule is only as effective as its position and its precision. The two block rules were narrow, one by protocol and destination and one by protocol and port, so each removed exactly one service and left DNS and HTTPS available. Because the evaluation runs from the top and stops at the first matching Quick rule, both rules had to be placed above Default allow LAN to any, otherwise the broad allow rule would have matched first and the traffic would have continued to pass.

The three views of the evidence agree. The client saw a failed ping and a timed-out HTTP request, the live log named the rule that blocked each packet, and Wireshark showed the same packets

leaving the client without a reply. The block action drops packets silently, which is why the client saw only retransmitted SYN packets and no error message. From a security view, narrow logged rules give both control and an audit trail, and restoring the original state afterwards keeps the laboratory reliable for the next activity.

ANALYSIS AND FINDINGS

Table 6: Verification summary

Test	Method	Result
Baseline	ping, getent, curl HTTP and HTTPS	All four tests succeeded before any rule was added.
Rule order	Firewall > Rules > LAN	Both LAB2 rules above Default allow LAN to any.
ICMP block	ping -c 4 1.1.1.1	100% packet loss; DNS and HTTPS still worked.
HTTP block	curl --max-time 10 -I http://example.com	Timed out; HTTPS returned headers.
Log evidence	Firewall > Log Files > Live View	Blocked entries named the LAB2 rules.
Packet evidence	Wireshark filters	No ICMP replies; port 80 SYN retransmitted; port 443 completed.
State and NAT	Diagnostics > States; Source NAT	Permitted HTTPS state and translated WAN state seen; NAT mode unchanged.
Restoration	Rules disabled; baseline repeated	ping, HTTP and HTTPS all succeeded again.

All planned tests behaved as expected: each block rule stopped only its own traffic, the log and the packet capture agreed with the client results, and connectivity was fully restored after the rules were disabled.

CHALLENGES AND SOLUTIONS

No configuration errors occurred, but the logs and the capture showed other port 80 connections from the client (for example to 185.125.190.99 and 91.189.91.61) besides the curl test. The Live View was therefore filtered by the LAB2 label and the test entries were identified by time and destination, which keeps the evidence tied to the commands that were run. The table below lists the faults most likely to occur in this lab and the corrective action for each.

Table 7: Common faults and corrective actions

Symptom	Likely cause	Solution
Traffic is still allowed	The block rule is below the allow rule, is on the wrong interface, or the changes were not applied.	Move the rule above Default allow LAN to any, confirm the LAN interface, and select Apply changes.
All traffic stops	The rule is broader than intended.	Disable it immediately, recheck protocol, destination and port, then re-enable.
No log entries appear	Logging is not enabled or the Live View filter is too narrow.	Enable logging in the rule, clear the filter and generate fresh traffic.
HTTP test succeeds	A redirect was followed, or the rule uses the wrong port.	Use curl without -L, check TCP destination port 80 and open a new connection.
Access does not return after disabling the rules	Old firewall states, or another configuration problem.	Apply changes, clear only the relevant states, repeat the Laboratory 1 baseline and check that the rules are disabled.

CONCLUSION

Two narrow firewall rules were created on the OPNsense LAN interface, placed above the default allow rule and tested from the Ubuntu client. The ICMP rule stopped pings to 1.1.1.1 while DNS and HTTPS continued, and the HTTP rule stopped traffic on port 80 while port 443 continued. The OPNsense live log, the Wireshark capture and the state table gave matching evidence, and normal connectivity was restored by disabling the test rules. The laboratory shows that careful rule order, narrow matching and logging make firewall behaviour predictable and provable.

WHAT WAS LEARNED

This lab taught me that a firewall decision depends on rule order as much as on rule content, and that the first matching rule wins. I learned to build a rule field by field so that it blocks only what I intend, and to prove its effect from three sides: the client result, the firewall log and the packet capture. Seeing a silent drop as repeated SYN packets in Wireshark, compared with a clean handshake on port 443, made the difference between blocked and permitted traffic real to me. I also learned that testing a baseline first and restoring it afterwards are part of professional practice, because they make every result trustworthy and keep the environment safe for the next task.

RECOMMENDATIONS

- Record a full baseline before changing any rule, and stop if the baseline fails, so that later faults can be tied to the new rule.

- Make test rules as narrow as possible, enable logging on them, and place them above the broad allow rule before applying changes.
- Disable, rather than delete, temporary rules until the evidence is complete, then confirm that the baseline tests pass again.

ANALYSIS QUESTIONS

1. Why must the specific block rules be placed above the broad allow rule?

OPNsense reads the rules of an interface from top to bottom and the first matching Quick rule decides the packet. The Default allow LAN to any rule matches every packet from the LAN network, so a block rule below it would never be reached and the traffic would still be allowed.

2. Which five packet attributes are most useful when explaining a firewall decision?

The source IP address, the destination IP address, the protocol, the destination port (where the protocol has ports) and the action together with the rule label. The interface and direction identify where the packet was seen and the timestamp places it in time.

3. Why did blocking ICMP not block HTTPS?

The rule matched only protocol ICMP to 1.1.1.1. HTTPS uses TCP port 443 and a different destination, so it did not match the rule and was handled by Default allow LAN to any.

4. What difference did you observe between the blocked TCP port 80 traffic and permitted TCP port 443 traffic?

On port 80 the client sent a SYN, received no SYN-ACK and retransmitted the SYN, so no handshake completed. On port 443 the three-way handshake completed and encrypted TLS traffic followed.

5. What role does outbound NAT play when icdfa-nslab-client-v1 uses a private IPv4 address?

Private addresses such as 10.10.10.201 cannot be routed on the internet. Outbound NAT rewrites the source address to the WAN address 10.0.2.15 and keeps a state so the replies can be translated back and delivered to the client.

6. Why is restoring the original state an important part of a controlled security laboratory?

A known-good state makes the next activity reliable, because leftover rules could cause false faults. It also supports change control and shows that the test changes were temporary and under control.

REFERENCES

International Cybersecurity and Digital Forensics Academy. (2026). WADF105 - Network Security Fundamentals: Lab 2 OPNsense Firewall Policy Testing, Logging and Packet Analysis, Official Laboratory Brief.

Deciso B.V. (n.d.). OPNsense Documentation. https://docs.opnsense.org

Wireshark Foundation. (n.d.). Wireshark User's Guide. https://www.wireshark.org/docs

Postel, J. (1981). RFC 792: Internet Control Message Protocol. IETF.

Postel, J. (1981). RFC 793: Transmission Control Protocol. IETF.

Mockapetris, P. (1987). RFC 1035: Domain Names - Implementation and Specification. IETF.